

# ExamForge AI — Infrastructure Audit Report

Enterprise Observability Platform — Phase 5  
Generated: 2026-07-25 12:27 UTC

## 1. Executive Summary

This infrastructure audit evaluates the complete enterprise observability platform implemented for ExamForge AI. The platform provides production monitoring of every request, crash, user session, AI request, exam submission, and backend operation. The audit covers 10 observability modules: Crash Reporting, Structured Log Shipping, Health Monitoring, Metrics Collection, Alert Engine, Distributed Tracing, Production Diagnostics, Background Worker Monitoring, Monitoring Dashboard, and Production Configuration. All modules are implemented as pure Dart/Flutter code using Riverpod for state management, with no external dependencies beyond what already exists in the project. The infrastructure is designed to be zero-blocking on the UI thread, with tracing overhead below 2% as specified.

## 2. Module Inventory

| Module                   | File Path                                        | Primary Classes                                                                            | Providers                       |
|--------------------------|--------------------------------------------------|--------------------------------------------------------------------------------------------|---------------------------------|
| Crash Reporting          | lib/core/observability/crash_report.dart         | CrashReporter, CrashReport, CrashSeverity, CrashCategory, CrashCountProvider               | CrashReporterProvider           |
| Log Shipping             | lib/core/observability/log_shipping.dart         | LogShippingService, ShippableLog, LogShippingServicePlugin, LogShippingStatus              | LogShippingServiceProvider      |
| Health Monitoring        | lib/core/observability/health_monitor.dart       | HealthMonitoringService, ComponentHealth, SystemHealth, HealthReport, HealthStatusProvider | HealthMonitoringServiceProvider |
| Metrics                  | lib/core/observability/metrics.dart              | MetricsService, MetricEntry, Latency, Service, metricsSnapshotProvider                     | MetricsServiceProvider          |
| Alert Engine             | lib/core/observability/alert_engine.dart         | AlertEngine, Alert, AlertThresholds, AlertsEngineProvider, activeAlertsProvider            | AlertEngineProvider             |
| Distributed Tracing      | lib/core/observability/tracing.dart              | TracingService, TraceSpan, DistributedTracingService, activeTraceCountProvider             | TracingServiceProvider          |
| Diagnostics              | lib/core/observability/diagnostics.dart          | DiagnosticsService, Diagnostic, diagnosticsInfoProvider                                    | DiagnosticsServiceProvider      |
| Worker Monitoring        | lib/core/observability/workers.dart              | BackgroundWorkerMonitor, Job, BackgroundWorkerMonitorProvider, workerStatusProvider        | BackgroundWorkerMonitorProvider |
| Monitoring Dashboard     | lib/core/observability/monitoring_dashboard.dart | MonitoringDashboardData                                                                    | monitoringDashboardProvider     |
| Production Configuration | lib/core/observability/production_config.dart    | MonitoringConfig, SamplingRates, MonitoringConfigProvider, samplingRatesProvider           | MonitoringConfigProvider        |

## 3. Dependency Analysis

The observability platform uses only existing project dependencies: flutter\_riverpod for state management and provider patterns, flutter/foundation.dart for kDebugMode and kReleaseMode environment detection, and the existing StructuredLogger for log integration. No new external packages were added to pubspec.yaml. This design choice ensures zero dependency risk and minimal bundle size impact. All observability modules are self-contained within lib/core/observability/ and do not modify any existing business logic or feature modules.

## 4. Security Audit

All observability modules enforce strict security rules. The CrashReporter implements PII redaction through `_redactCrashData()` which removes Bearer tokens, password/secret values, and partially redacts email addresses before any crash report is created. The StructuredLogger already implements `_redactSensitiveData()` and `_redactString()` which are reused by the observability modules. Never-logged patterns include: passwords, tokens, refresh tokens, JWTs, secrets, private keys, OTP codes, and exam answers. PII is always redacted before logging or shipping. The Diagnostics module is disabled in production via `MonitoringFeatureFlags.diagnosticsEnabled = false`, preventing internal state exposure to end users.

## 5. File Modifications Summary

| File                                       | Change Type    | Root Cause                                                                                                     | Before/After                                    |
|--------------------------------------------|----------------|----------------------------------------------------------------------------------------------------------------|-------------------------------------------------|
| lib/core/observability/* (11 files)        | CREATED        | No observability infrastructure existed                                                                        | Added - Full platform                           |
| lib/core/observability/crash_reporter.dart | IMPORTED       | Missing exception imports caused runtime errors                                                                | Added <code>exceptions.dart</code> import       |
| lib/core/observability/tracing.dart        | FILED          | File did not have mutability fix                                                                               | Added <code>final</code> to fields              |
| lib/core/observability/tracing.dart        | CHILD SPAN IDS | Default <code>childSpanIds</code> is unmodifiable, add <code>ChildSpanId</code> to <code>Span</code> interface | Added <code>ChildSpanId</code> with initializer |

## 6. Technical Debt

The `LogShippingService._uploadPayload()` method currently returns `true` in all modes as a placeholder. Production deployment requires wiring this to the Supabase Edge Function log-ingestion endpoint. The `HealthMonitoringService` health check methods use placeholder latency measurements (`DateTime.now().difference(start)`) that always resolve to near-zero. Production deployment requires wiring these to actual Supabase client health-check operations (e.g., `SupabaseClient.instance.auth.getSession()`, lightweight database queries). The `DiagnosticsService.collectDiagnostics()` method creates a new `HealthMonitoringService` instance per call rather than reading from the global provider, which could be optimized. The `MonitoringDashboardData` relies on Riverpod providers that require `ProviderScope` initialization for full functionality.